```
# Step 1: Import Necessary Libraries
import numpy as np

# Step 2: Initialize the Perceptron
# Input data (Manual dataset - AND gate example)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Output labels
y = np.array([0, 0, 0, 1]).reshape(4, 1)

# Initialize weights and bias
weights = np.array([[0.1], [0.3]])
bias = 0.2

print("Initial Weights:\n", weights)
print("Initial Bias:", bias)

# Step 3: Define Activation Function
def sigmoid_func(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def der(x):
    return x * (1 - x)

# Step 4: Training Data already defined above

# Step 5: Learning Rate and Epochs
alpha = 0.05
epochs = 15000

# Step 6: Training the Perceptron
for epoch in range(epochs):

    # Forward propagation
    weighted_sum = np.dot(X, weights) + bias
    output_pred = sigmoid_func(weighted_sum)

    # Error calculation
    error = output_pred - y

    # Backpropagation
    d_output = error * der(output_pred)

    # Weight update
    weights -= alpha * np.dot(X.T, d_output)

    # Bias update
    bias -= alpha * np.sum(d_output)

# Final weights and bias
print("\nFinal Weights:\n", weights)
print("Final Bias:", bias)

# Step 7: Prediction Function
def predict(input_data):
    result = np.dot(input_data, weights) + bias
    return sigmoid_func(result)

# Step 8: Evaluate the Model (Predictions)

print("\n--- Predictions ---")
```

```
test_inputs = [
    [1, 0],
    [1, 1],
    [0, 0],
    [0, 1]
]

for inp in test_inputs:
    res = predict(np.array(inp))
    print(f"Input: {inp} -> Output: {res}")
```

```
Initial Weights:
 [[0.1]
 [0.3]]
Initial Bias: 0.2

Final Weights:
 [[5.13607423]
 [5.13607425]]
Final Bias: -7.7984793971029855

--- Predictions ---
Input: [1, 0] -> Output: [0.06522853]
Input: [1, 1] -> Output: [0.92227519]
Input: [0, 0] -> Output: [0.00041019]
Input: [0, 1] -> Output: [0.06522853]
```